Host your Python agents on Microsoft Foundry

Apr 27: **Microsoft Agent Framework**

Apr 29: **LangChain and LangGraph**

Apr 30: **Quality and safety evals**

Register at aka.ms/AgentsOnFoundry/series

Microsoft

Host your agents on Foundry

LangChain and LangGraph

aka.ms/foundryhosted/slides/langchain

Pamela Fox

Python Cloud Advocate

www.pamelafox.org

Today we'll cover…

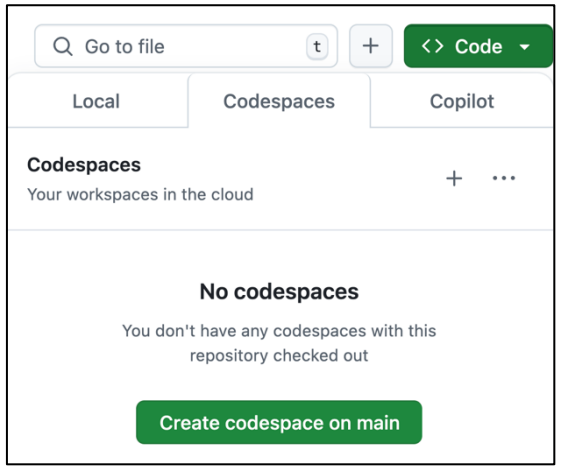
- Agents 101
- Building agents with LangChain
- Foundry models for inference
- Foundry IQ for knowledge retrieval
- Foundry tools for cloud-hosted agentic tools
- Hosting agents on Foundry Agent Service
- Building multi-agent workflows
- Hosting workflows on Foundry Agent Service

Want the code?

1. Open this GitHub repository:

aka.ms/foundry-hosted-langchain-demos

2. Use "Code" button to create a GitHub Codespace:

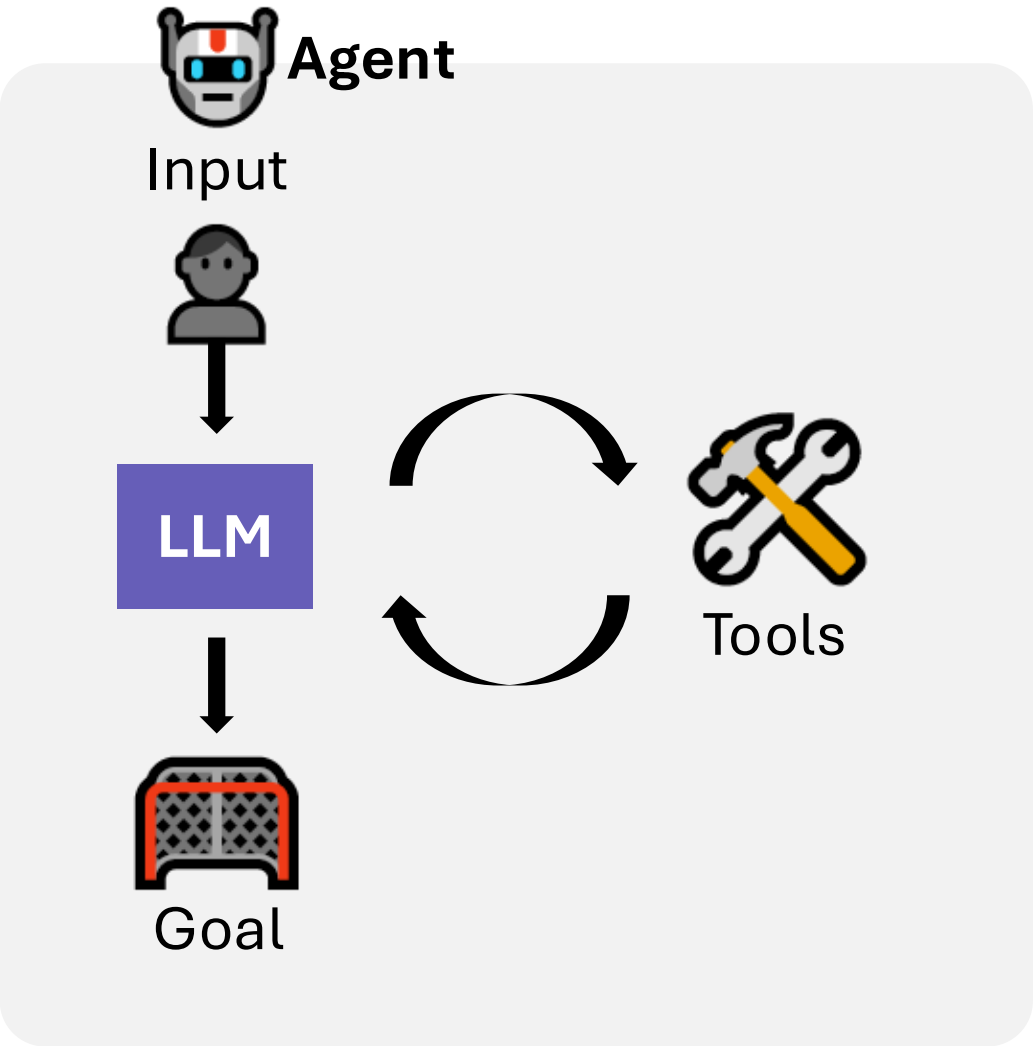


3. Wait a few minutes for Codespace to start up ⏱️

⚠️ Most code samples require deployment and an Azure account.

Agents 101

What is an agent?



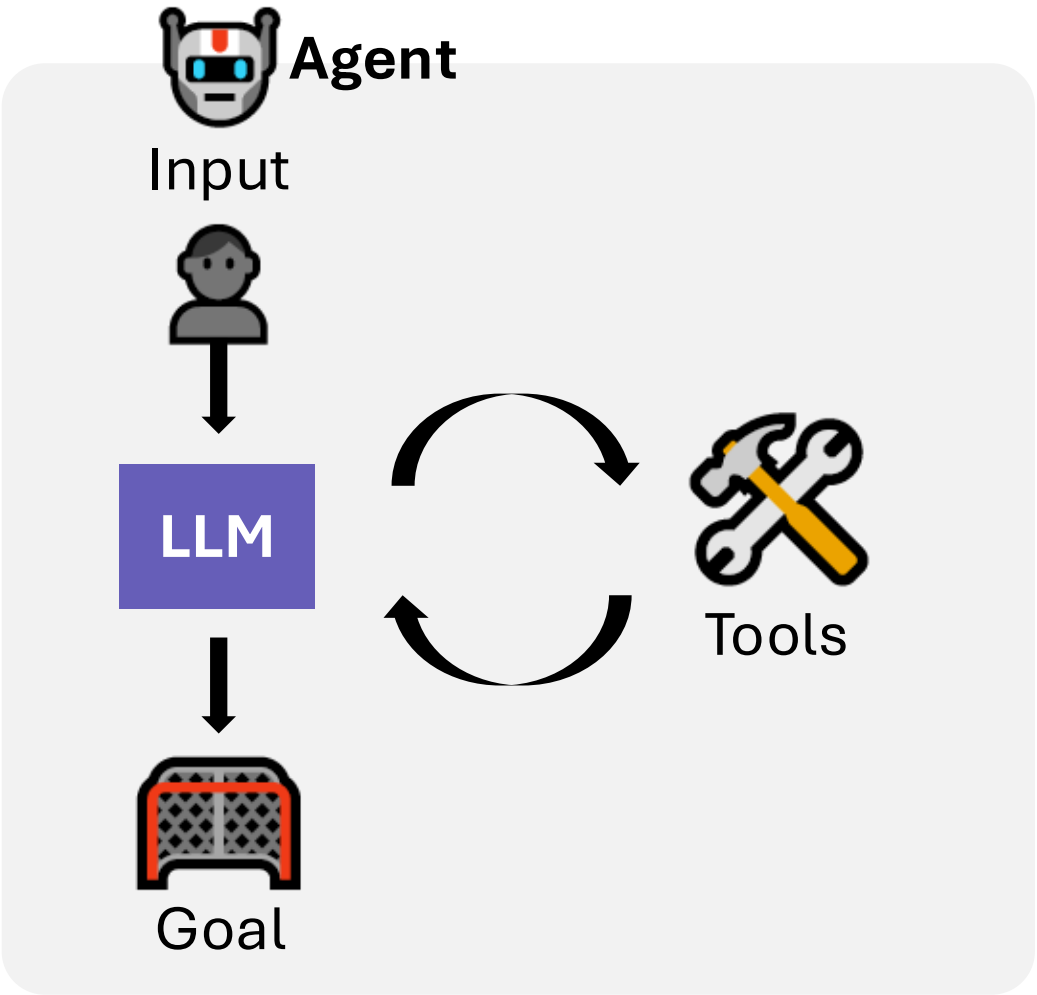
An **AI agent** uses an **LLM** to run **tools** in a loop to achieve a **goal**.

Agents are often augmented by:

- Context
- Memory
- Planning
- Humans

Dive into advanced agentic features in our previous Python + Agents series:
aka.ms/pythonagents/rewatch

The agentic loop in action



The agent keeps calling tools until goal is met:

Python AI agent frameworks

A framework makes it easier to handle tool calling and context management:

Framework	Description
agent-framework	A framework from Microsoft with support for agentic patterns and full integration with Azure offerings. (Successor to autogen and semantic-kernel)
langchain v1	An agent-centric framework built on top of Langgraph, with optional Langsmith monitoring
pydantic-ai	A flexible framework designed for type safety and observability (Logfire/OTel), from the creators of Pydantic

There are many frameworks, but these are popular Python frameworks.

LangChain and LangGraph

Open-source framework for building and orchestrating intelligent AI agents and workflows

Two complementary open-source libraries

LangChain

Composable framework with 100+ integrations to quickly build agents, chatbots, and RAG pipelines.

JS

LangGraph

Graph-based orchestration for stateful, multi-step agents with cycles, branching, and human-in-the-loop.

JS

Agents • Orchestration • Memory • State • Cloud-agnostic

langchain.com

Installing Langchain for Python

Install the main Langchain package:

```
>> pip install langchain
```

Install the main Langgraph package:

```
>> pip install langgraph
```

Install the provider-specific package for OpenAI:

```
>> pip install langchain-openai
```

Many other model provider packages are also available.

📌 Always pin your versions and use a lock file!

📎 See example: pyproject.toml

A simple local agent with tools

langchain/langgraph

```
client = ChatOpenAI(
    base_url="http://localhost:11434/v1/",
    model="qwen3.5:4b",
    api_key="no-key-needed")

@tool
def get_enrollment_deadline_info() -> dict:
  """Return enrollment timeline details for health insurance plans."""
  return {"enrollment_opens": "2026-11-11", "enrollment_closes": "2026-11-30"}

agent = create_agent(model=client,
  system_prompt=f"You're an HR helper. Today's date is {date.today().isoformat()}. ",
  tools=[get_enrollment_deadline_info])

result = await agent.ainvoke({"messages": [{"role": "user", "content": "What PerksPlus
benefits are there, and when do I need to enroll by?",}]})["messages"][-1]
response = result["messages"][-1]
print(response.text)
```

Full example: [agents/stage0_local_model.py]

Building agents on Foundry

Microsoft Foundry

The end-to-end platform to design, customize, and deploy AI-driven applications at scale.

Models

Access frontier models like Opus 4.6 and GPT-5.4.

IQ

Ground LLMs in domain data, SharePoint, web search, and more.

Agent Service

Host agents with memory, tools, observability, and evaluations.

Tools

Connect to 1,400+ built-in service connections and MCP tools.

Machine Learning

Train and deploy custom ML models for specific business needs.

Control Plane

Leverage a complete signals management layer with Microsoft Security integrations

Microsoft Agent 365 | Microsoft Defender | Microsoft Purview | Microsoft Entra

Security, compliance, and governance

Using Langchain with Foundry

Install our Microsoft-supported Langchain package for Azure AI / Foundry:

```
>> pip install langchain-azure-ai
```

Install the `opentelemetry` extra for emitting traces to App Insights:

```
>> pip install "langchain-azure-ai[opentelemetry]"
```

Install the `tools` extra for Foundry toolbox integration:

```
>> pip install "langchain-azure-ai[tools]
```

For other Azure integrations, check out these sibling packages:

```
langchain-azure-cosmosdb          langchain-azure-postgresql   langchain-azure-storage
langchain-azure-dynamic-sessions  langchain-sqlserver
```

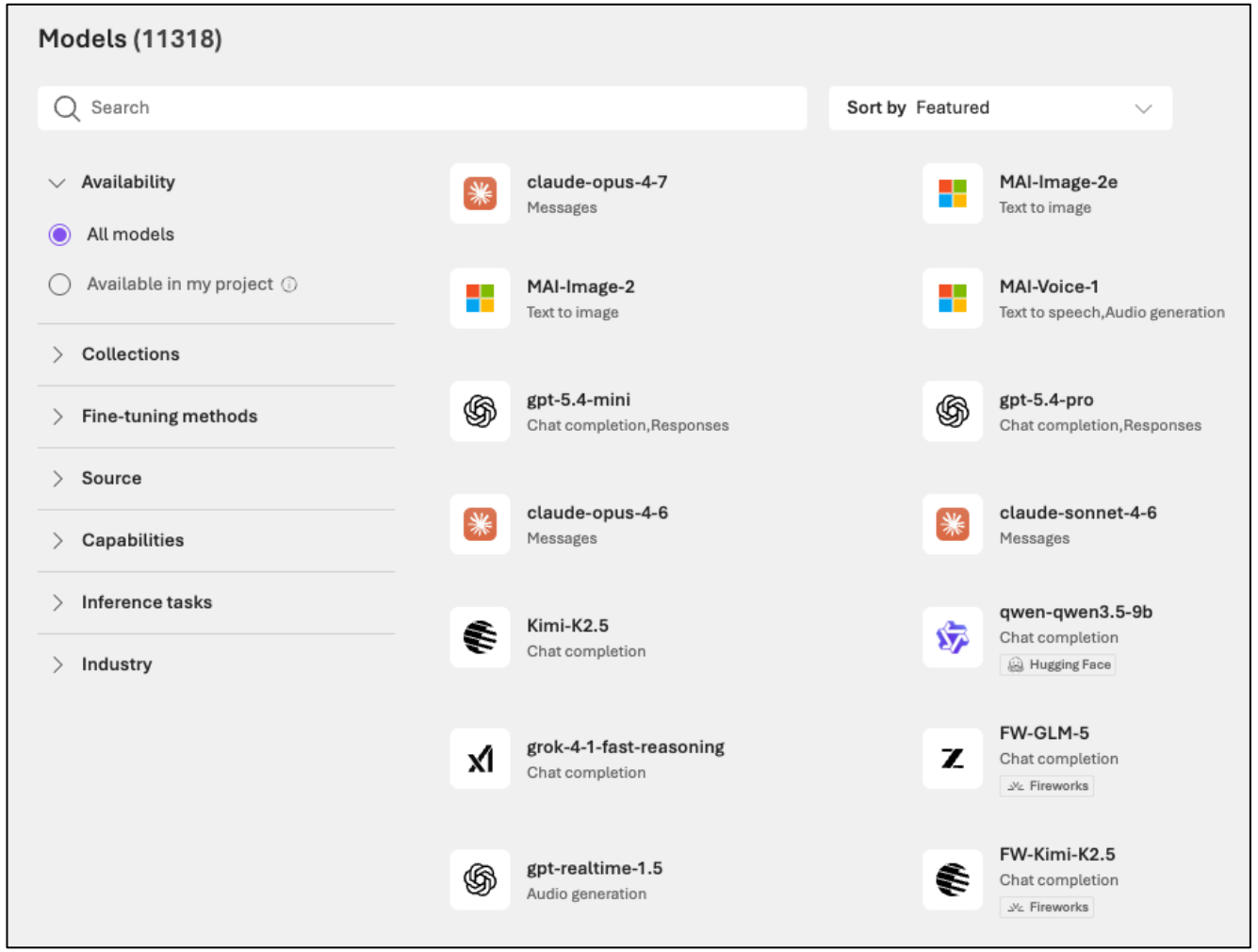
What does langchain-azure-ai provide?

Feature	Description
Foundry Tools	Connect LangChain agents to built-in tools from Foundry tools catalog and Foundry toolboxes.
Foundry Agent Service integrations	Connect hosted Foundry agents into LangGraph as reusable nodes, so you can delegate tasks to managed agents and compose multi-agent workflows with Foundry project auth/tooling.
OpenTelemetry tracing	Export LangChain and LangGraph traces to Application Insights
Document ingestion and RAG utilities	Use document loaders, retrievers, vector store helpers, and Content Understanding support for grounding scenarios.
AI Content Safety middleware	Add guardrails as middleware (text/image moderation, prompt-injection detection, protected material checks, groundedness checks) directly in LangChain/LangGraph flows.

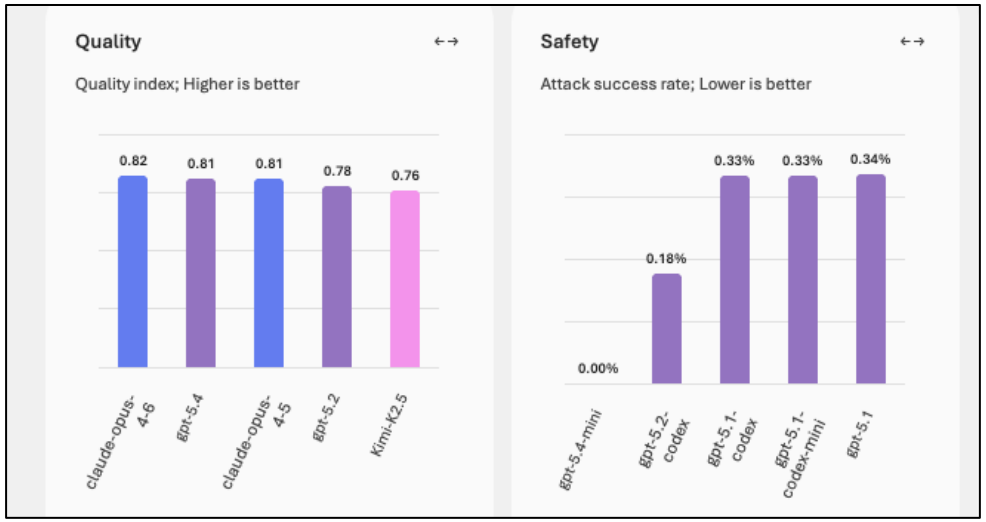
And more!

Foundry Models

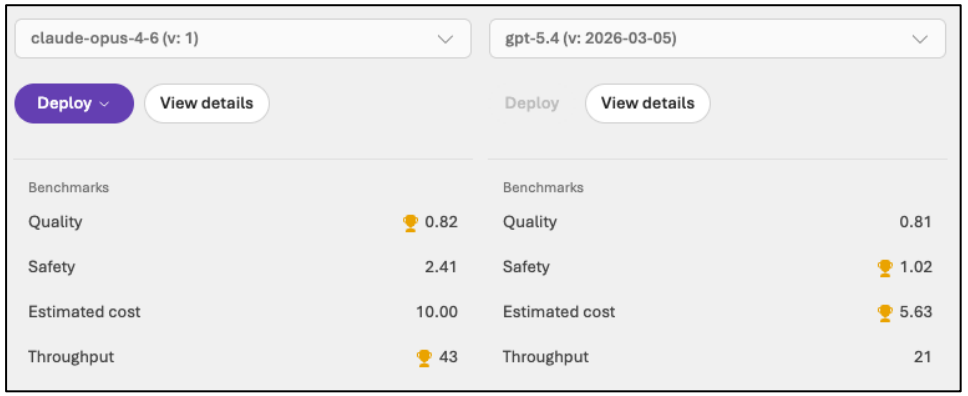
11,000+ models, both open-source and frontier models.



See overall metrics on leaderboards:



Compare models side-by-side:



An agent with Foundry models

langchain/langgraph

```
client = ChatOpenAI(
  base_url=f"{os.environ['AZURE_OPENAI_ENDPOINT']}/openai/v1/",
  api_key=token_provider,
  model=os.environ["AZURE_AI_MODEL_DEPLOYMENT_NAME"])
```

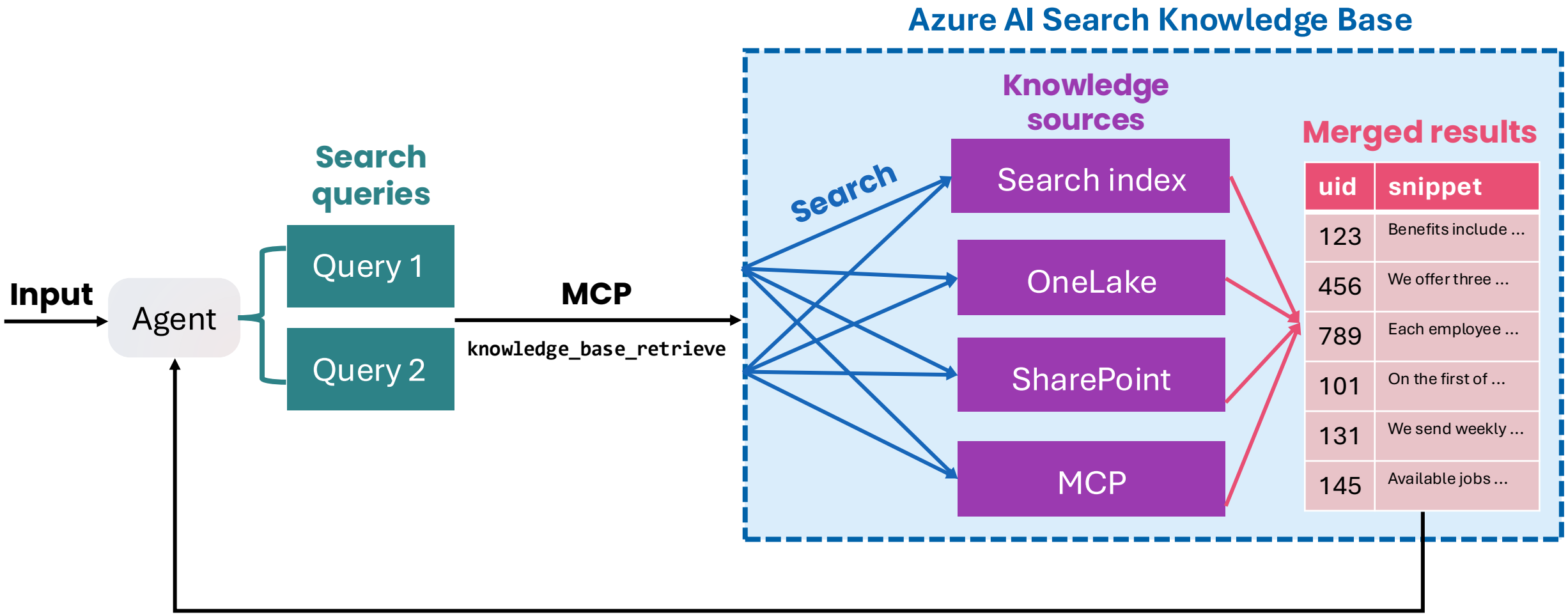
Only the client changes

```
@tool
def get_enrollment_deadline_info() -> dict:
"""Return enrollment timeline details for health insurance plans."""
 return {"enrollment_opens": "2026-11-11", "enrollment_closes": "2026-11-30"}

agent = create_agent(client=client,
  instructions= f"You're an HR helper. Today's date is {date.today().isoformat()}.",
  tools=[get_enrollment_deadline_info])

result = await agent.ainvoke({"messages": [{"role": "user", "content": "What PerksPlus
benefits are there, and when do I need to enroll by?",}]}))
response = result["messages"][-1]
print(response.text)
```

Full example: agents/stage1_foundry_model.py

Foundry IQ

Generate unified context using agentic retrieval across multiple knowledge sources (indexed and remote)



Adding Foundry IQ as MCP tool

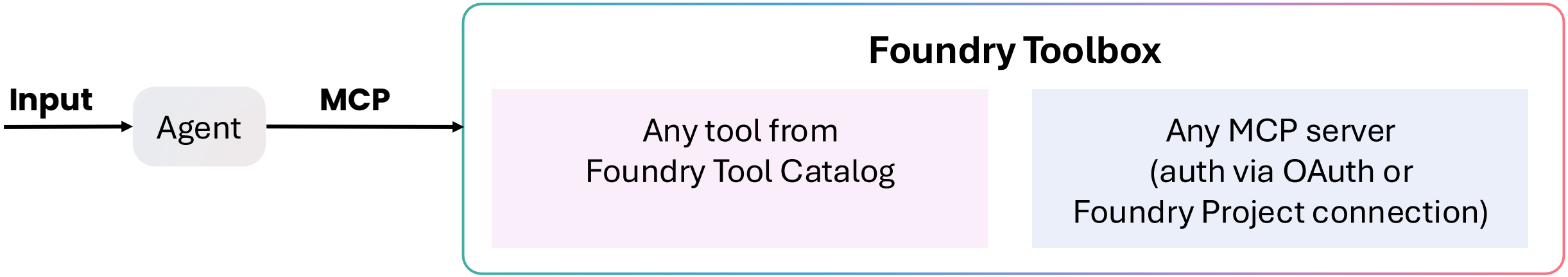
langchain/langgraph

```
mcp_url = (f"{os.environ['AZURE_AI_SEARCH_SERVICE_ENDPOINT'].rstrip('/')}"
           f"/knowledgebases/{os.environ['AZURE_AI_SEARCH_KNOWLEDGE_BASE_NAME']}"
           f"/mcp?api-version=2025-11-01-Preview")

kb_client = MultiServerMCPClient({
     "knowledge-base": {
         "url": mcp_url,
         "transport": "streamable_http",
         "auth": _AzureTokenAuth(search_token_provider),
     }
 })
 kb_tools = await kb_client.get_tools()

 agent = create_agent(
     model=client,
     tools=[get_enrollment_deadline_info, *kb_tools],
     system_prompt=f"You're an HR helper. Today is {date.today().isoformat()}. ",)
```

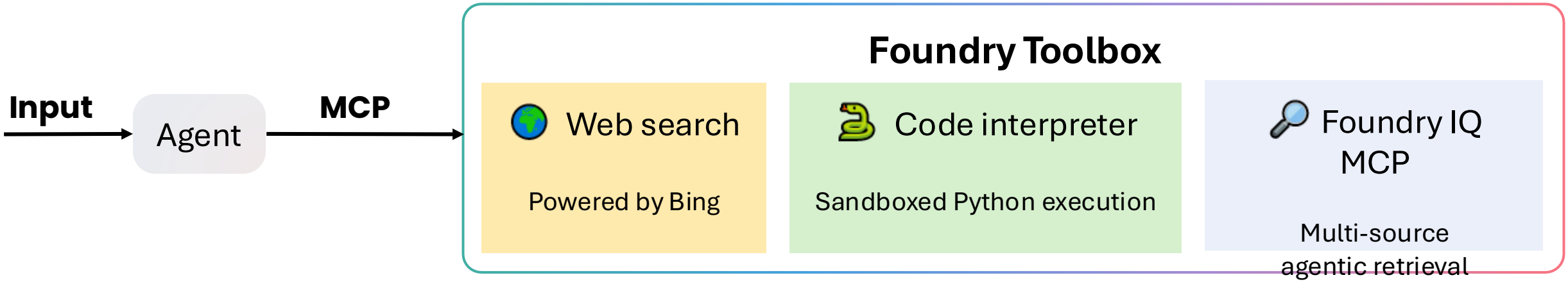
Full example: agents/stage2_foundry_iq.py

Foundry Tools

Create a toolbox on Foundry composed of Foundry Tools and MCP servers, for use by any agent.



Today we'll build this toolbox:



Creating a toolbox from code

```
tools = [
  {"type": "web_search", "name": "web_search"},
  {"type": "code_interpreter", "name": "code_interpreter"},
  {
   "type": "mcp",
   "server_label": "knowledge-base",
   "server_url": 'https://${search_service}.search.windows.net/knowledgebases/'
                 ${knowledge_base_name}/mcp?api-version=2025-11-01-preview',
   "project_connection_id": 'kb-mcp-connection',
   "allowed_tools": ["knowledge_base_retrieve"]}
]

resp = httpx.post(f"{PROJECT_ENDPOINT}/toolboxes/{TOOLBOX_NAME}/versions",
  params={"api-version": "v1"}, json={"tools": tools})

httpx.patch(f"{PROJECT_ENDPOINT}/toolboxes/{TOOLBOX_NAME}",
  params={"api-version": "v1"}, json={"default_version": resp.json()["version"]})
```

Full code: [create-toolbox.py]

KB Connection made in: [azure_ai_search.bicep]

Equip agent with toolbox

langchain/langgraph

```
from langchain_azure_ai.tools import AzureAIProjectToolbox

toolbox = AzureAIProjectToolbox(
    toolbox_name=TOOLBOX_NAME,
    credential=DefaultAzureCredential())
toolbox_tools = await toolbox.get_tools()

agent = create_agent(
     model=client,
     tools=[
         get_enrollment_deadline_info,
         *toolbox_tools
     ])
```

Full code: agents/stage3_foundry_toolbox.py

Foundry Agent Service

Host your agents with production-grade security, reliability, and governance.

Prompt Agents
Define instructions and tools

Hosted Agents
Use your favorite agent framework

Foundry Tools

Foundry IQ

Foundry Memory
Managed Memory
Managed Conversations
BYO-Memory Store

Foundry Models

Foundry Control Plane

- Controls
- Observability
- Security
- Fleet-wide Operations

Microsoft Agent 365

Microsoft Defender

Microsoft Entra

Microsoft Purview

Hosted BYO Agents in Foundry Agent Service

Serve any agent using the Responses API or more generic invocations API.

Foundry Agent Service

Identity · Endpoint · State Scaling · Observability

Container (in Azure Container Registry)

Responses API Adapter

`/agents/{name}/endpoint/protocols/openai/responses`

Your code

.NET

Java

JS

Hosted BYO Agents in Foundry Agent Service

A community adapter bridges LangGraph to the Responses API protocol

Foundry Agent Service

Container (in Azure Container Registry)

`Dockerfile` `agent.yaml`

Responses API Adapter

```
from langchain_azure_ai_runtime import AzureAIResponsesAgentHost
```

Your code

```
agent = create_agent(
     model=ChatOpenAI(...),
     tools=[get_enrollment_deadline_info, *toolbox_tools]
 )
```

```
host = AzureAIResponsesAgentHost(graph=agent, stream_mode="messages",
     responses_history_count=20,)
host.run()
```

How the adapter works

langchain/langgraph

input

Responses API	LangGraph/LangChain
`MessageContentInputTextContent`	`HumanMessage`
`MessageContentOutputTextContent`	`AIMessage`
`system / developer role`	`SystemMessage`

output

Responses API	LangGraph/LangChain
`function_call`	`AIMessage(tool_calls=[...])`
`output_text`	`AIMessage(content="...")`

A Foundry hosted LangGraph agent

langchain/langgraph

```python
from langchain.agents import create_agent
from langchain_azure_ai.callbacks.tracers import enable_auto_tracing
from langchain_azure_ai.tools import AzureAIProjectToolbox
from langchain_openai import ChatOpenAI
from vendor.langchain_azure_ai_runtime import AzureAIResponsesAgentHost

enable_auto_tracing(auto_configure_azure_monitor=True, agent_id="hr-agent")

llm = ChatOpenAI(base_url=f"{PROJECT_ENDPOINT}/openai/v1", api_key=token_provider,
     model=MODEL_DEPLOYMENT_NAME,
     use_responses_api=True,)

 agent = create_agent(model=llm,
     tools=[get_enrollment_deadline_info, get_current_date, *toolbox_tools],
     system_prompt="You are an internal HR helper.",)

 host = AzureAIResponsesAgentHost(graph=agent, stream_mode="messages")
 host.run()
```

Full example: [agents/stage4_foundry_hosted.py]

Deploy agent to Foundry with azd

```
>> azd auth login
>> azd up
```

What this does:

1. Provision infrastructure (AI project, model)
2. Builds image from Dockerfile (using ACR remote build)
3. Push image to Azure Container Registry
4. Deploys new agent version to Foundry

All based off these project files:

azure.yaml

```yaml
name: ai-foundry-starter-basic
services:
 hosted-langchain-agent:
  project: ./agents
   host: azure.ai.agent
   language: docker
   docker:
    remoteBuild: true
   config:
    startupCommand: python
stage4_foundry_hosted.py
```

Dockerfile

```dockerfile
FROM python:3.12-slim
WORKDIR /app
COPY --from=ghcr.io/astral-sh/uv:latest /uv /bin/uv
COPY pyproject.toml uv.lock ./
RUN uv sync --locked --no-dev
COPY stage4_foundry_hosted.py .
ENV PATH="/app/.venv/bin:$PATH"

EXPOSE 8088

CMD ["python", "-u", "stage4_foundry_hosted.py"]
```

agent.yaml

```yaml
kind: hosted
name: hosted-langchain-agent
protocols:
    - protocol: responses
      version: 1.0.0
resources:
  cpu: '0.25'
  memory: '0.5Gi'
environment_variables:
    ...
```

Interact with Foundry agent in playground

Microsoft Foundry / ai-project-pf-foundryhosted-agentframework

Search with AI (⌘K) | New Foundry | Home | Discover | Build | Operate | Docs

- Agents
- Models
- Fine-tune
- Tools
- Knowledge
- Data
- Evaluations
- Guardrails

hosted-agentframework-agent

v7 saved 4/20/2026 3:26 PM | Active | Publish

Playground | Traces | Monitor | Evaluation

Chat | Code | Metrics

Examples of what's covered (not exhaustive):

- Gym memberships
- Personal training sessions
- Yoga and Pilates classes
- Fitness equipment purchases
- Sports team fees
- Health retreats and spas
- Outdoor adventure activities (e.g., rock climbing, hiking, kayaking)
- Group fitness classes (e.g., dance, martial arts, cycling)
- Virtual fitness programs (online yoga/workout classes)
- Lessons/experiences like **skiing/snowboarding**, **scuba diving**, **surfing**, and **horseback riding** lessons

Not covered (examples):

- Non-fitness-related expenses
- Medical treatments/procedures
- Travel expenses (unless related to a fitness program)
- Food and supplements

Traces

Message the agent...

AI-generated content may be incorrect

Agent info

Entra agent identity
36ab37c4-30ed-4b45...

Entra agent blueprint
hosted-agentframew...

Code asset
crnivdj7gkpbmk2.azurecr.io/ai-foundry-starter-basic/...

Kind
hosted

Protocol
responses

Log stream | Files | Session: 0e2950ed0c942af2dacd...

Type to filter logs...

07:10:58.883 [stream] stderr 2026-04-21 14:10:58,883 INFO agent_framework: {'role': 'system', 'parts': [{'type': 'text', 'content': 'You are an internal HR helper focused on employee benefits and company information.\n Use the knowledge base tool to answer questions and ground all answers in provided context.\n You

Interact with Foundry agent from code

You can make direct HTTP calls to the Responses API endpoint:

```
curl -X POST "$BASE_URL/agents/$AGENT_NAME/endpoint/protocols/openai/responses?api-version=$API_VERSION" \
  -H "Authorization: Bearer $TOKEN" \
  -H "Content-Type: application/json" \
  -d '{ "input": "Hello! What can you do?", "store": true }'
```

Or use the azure-ai-projects package to send responses via an OpenAI client:

```
from azure.ai.projects import AIProjectClient

project = AIProjectClient(
  endpoint=os.environ["FOUNDRY_PROJECT_ENDPOINT"],
  credential=DefaultAzureCredential(),
  allow_preview=True)

openai_client = project.get_openai_client(agent_name="hosted-langchain-agent")
response = openai_client.responses.create(input="What PerksPlus benefits are there?")
print(response.output_text)
```

🔗 Full example: [agents/call_foundry_hosted.py]

Local development loop

Terminal 1

```
>> azd ai agent run

Using startup command: python stage4_foundry_hosted.py
Installing dependencies (pyproject.toml)...
  ✓ Dependencies installed (pyproject.toml)
Starting agent on http://localhost:8088
```

Terminal 2

```
>> azd ai agent invoke --local "What benefits are there?"

Target:       localhost:8088 (local)
Message:      "What PerksPlus benefits are there?"
Session:      6d872d38-a8e5-40bb-9813-34fb272c1785
Conversation: d47ecf0f-c2d6-485a-b628-561ec180bc92

[local] PerksPlus is Zava’s health & wellness reimbursement program. It lets employees
**expense up to $1,000** for eligible fitness/wellness-related items and activities.
```

👀 Observability

Observability with OpenTelemetry (OTel)

OTel standardizes how applications emit traces, metrics, and logs — making debugging and performance analysis consistent across languages and vendors.

Traces

```
0ms                                30ms
parent-span
└→ child-span
```

Operations composed of **spans** that show how a request moves through services, including timing, dependencies, and context propagation.”

Metrics

```
_/‾‾ cpu_usage: 30%
‾‾\__ latency_p95: 43ms
_/‾\__ errors: 0.2%
```

Numeric measurements such as CPU usage, request counts, latency, error rates, or any custom app metric.

Logs

```
INFO: 2025-12-05: Server running
      {"region": "westus"}
ERROR: 2025-12-05: User not found
      {"user_id": 123}
```

Structured log records with message, severity, time stamp, and contextual attributes.

Using OpenTelemetry with LangChain/LangGraph

LangChain Azure AI can emit OpenTelemetry traces for LangChain and LangGraph executions:

```
from langchain_azure_ai.callbacks.tracers import enable_auto_tracing

enable_auto_tracing(enable_content_recording=True)
```

🔗 Full example: [agents/stage4_foundry_hosted.py]

The emitted traces uses the standard "gen_ai" span attribute names and values for agent executions, LLM calls, and tool calls. Example tool call:

Span: `execute_tool get_weather`

attribute name	value
gen_ai.operation.name	execute_tool
gen_ai.tool.name	get_weather
gen_ai.tool.call.arguments	{"city": "Seattle"}
gen_ai.tool.call.result	{"temperature": 72, "description": "Sunny"}
gen_ai.tool.description	Returns weather data for a given city

Exporting OTel to App Insights

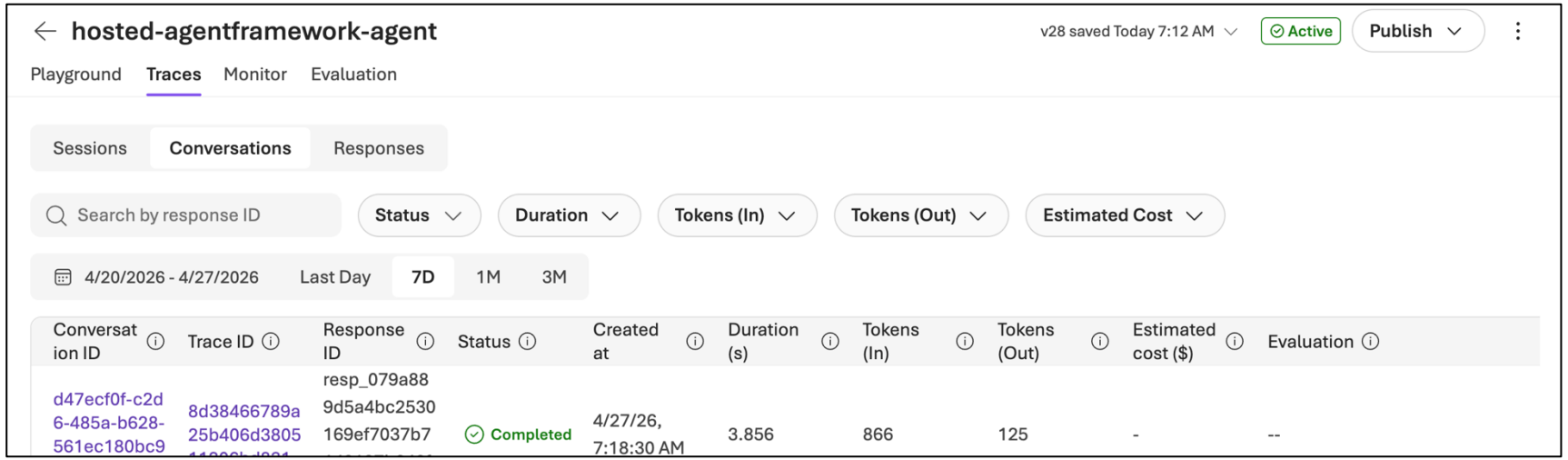
When the Foundry Project has an App Insights connection, Foundry agent service will auto-inject `APPLICATIONINSIGHTS_CONNECTION_STRING` environment variable.

Then `langchain-azure-ai` package configures the export to App Insights:

```
from langchain_azure_ai.callbacks.tracers import enable_auto_tracing

 enable_auto_tracing(
     auto_configure_azure_monitor=True,
     enable_content_recording=False,
     trace_all_langgraph_nodes=True,
     agent_id="hr-agent",)

 host = AzureAIResponsesAgentHost(
     graph=graph,
     stream_mode="messages",
     responses_history_count=20,
 )
 host.run()
```

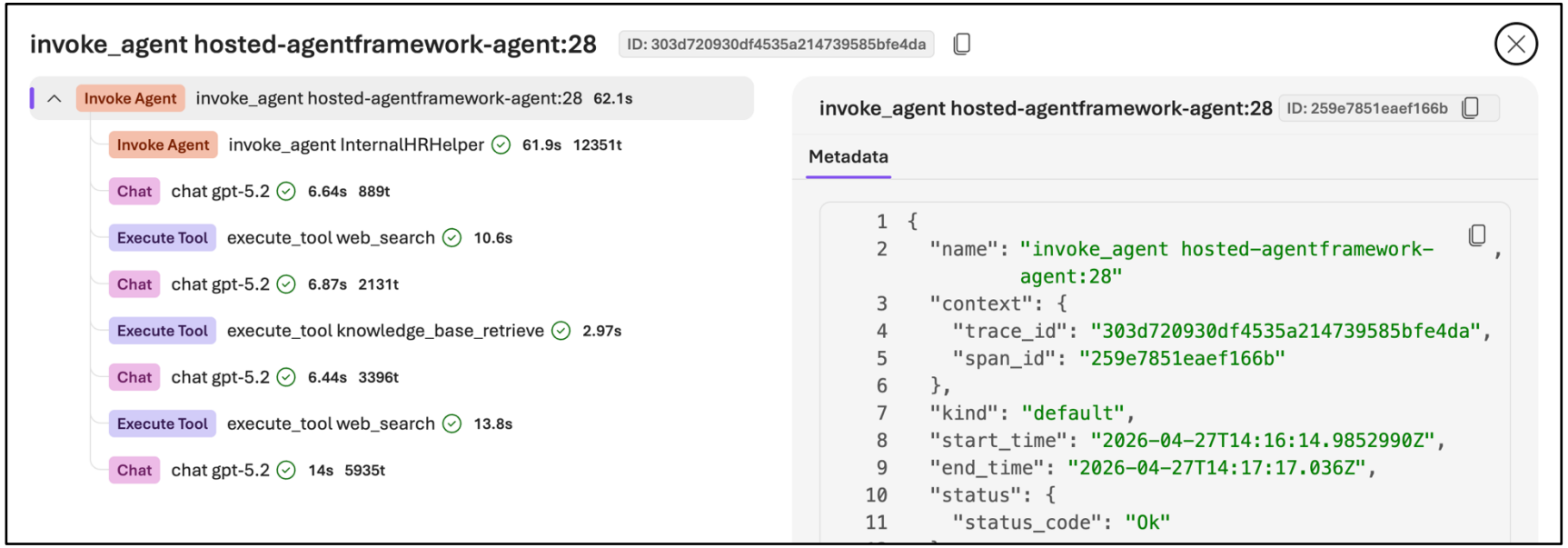
🔗 Full example: [agents/stage4_foundry_hosted.py]

Viewing agent traces in Foundry



From Foundry agent, select :

Traces > Conversations



See spans for:

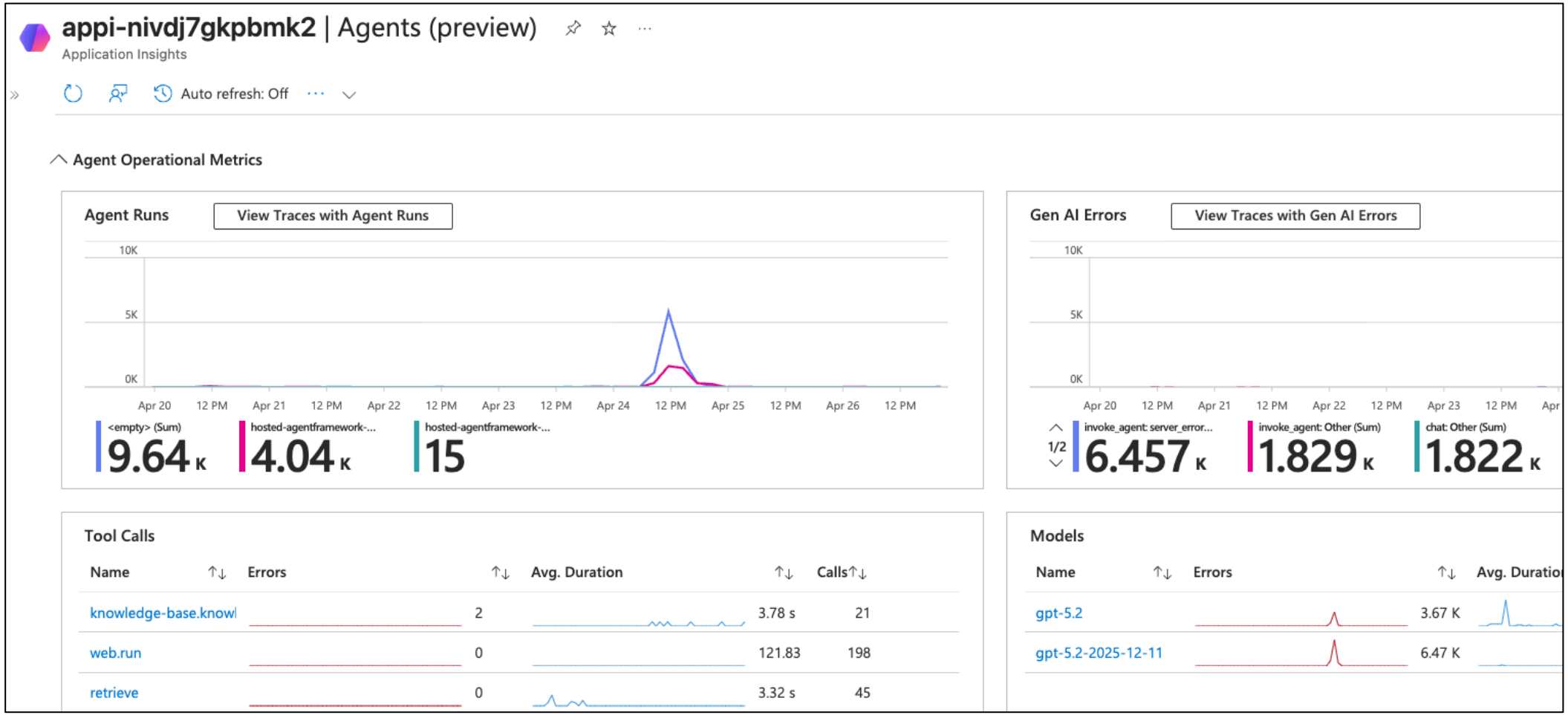
agent invocations,

LLM calls,

tool calls

Viewing agent traces in App Insights

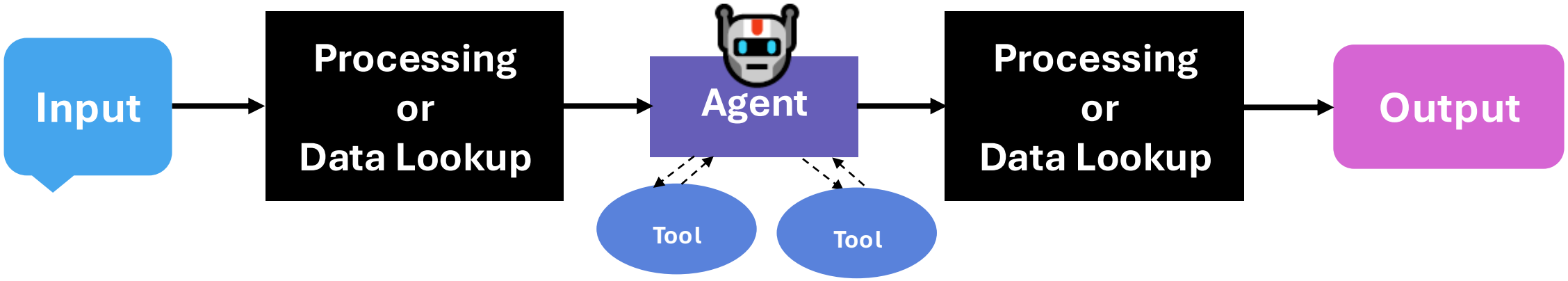
From Foundry agent, select Monitor > Open in Azure Monitor:



Multi-agent workflows

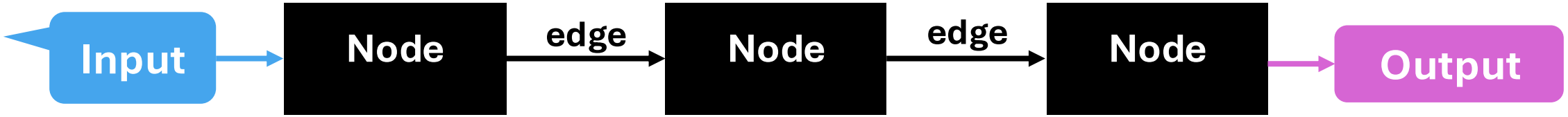
What is an agent workflow?

An agent workflow is any flow that involves an agent at some point, typically to handle decision making or answer synthesis.



Building workflows in LangGraph

A workflow is a StateGraph with node functions and edges between them



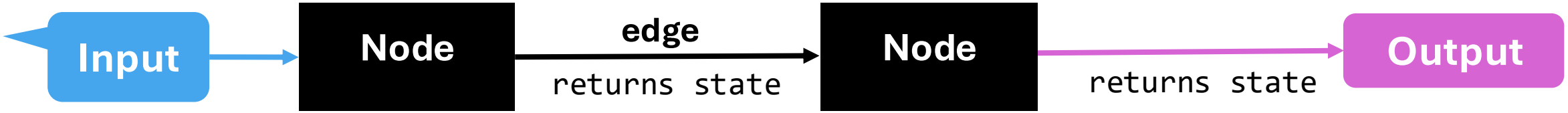
Each **node** is a function that **receives state**, modifies it, and **returns updated state**.

Edges define the flow, including **conditional edges** for branching.

[Workflows and agents - Docs by LangChain]

A simple workflow

langchain/langgraph



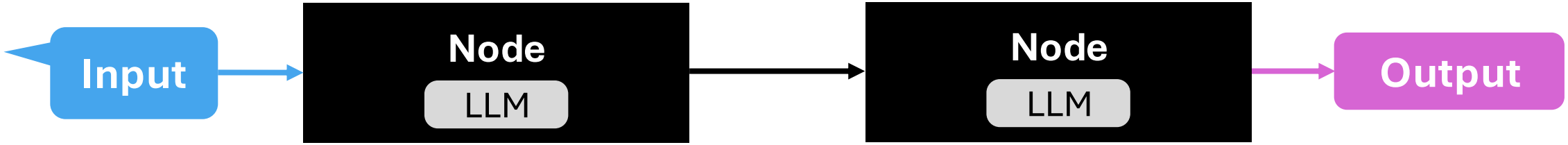

```python
class TextState(TypedDict):
    text: str

def upper_case(state: TextState) -> dict:
    return {"text": state["text"].upper()}

def reverse_text(state: TextState) -> dict:
    return {"text": state["text"][::-1]}

graph = (StateGraph(TextState).add_node(upper_case).add_node(reverse_text)
     .add_edge(START, "upper_case").add_edge("upper_case", "reverse_text")
     .add_edge("reverse_text", END)
     .compile())
result = graph.invoke({"text": "hello world"})
print(f"Output: {result['text']}")
```

Full example: [workflows/stage1_simple_nodes.py]

Using LLM calls inside workflows

langchain/langgraph

Each node can make LLM calls, just invoke the model inside the function




```
async def writer(state: TextState) -> dict:
     response = await llm.ainvoke([{"role": "system", "content": "You are a concise content
writer."}, {"role": "user", "content": state["text"]},])
     return {"text": response.content}

 async def formatter(state: TextState) -> dict:
     response = await llm.ainvoke([{"role": "system", "content": "Format text with emojis and
Markdown."}, {"role": "user", "content": state["text"]},])
     return {"text": response.content}

 graph = (StateGraph(TextState).add_node(writer).add_node(formatter)
     .add_edge(START, "writer").add_edge("writer", "formatter").add_edge("formatter", END)
     .compile())
```

Full example: workflows/stage2_agent_nodes.py

A Foundry hosted LangGraph workflow

Langchain/langgraph

```python
from langchain_azure_ai.callbacks.tracers import enable_auto_tracing
from langchain_azure_ai_runtime import AzureAIResponsesAgentHost

enable_auto_tracing(auto_configure_azure_monitor=True, agent_id="slogan-workflow")

graph = (StateGraph(MessagesState)
     .add_node(writer)
     .add_node(formatter)
     .add_edge(START, "writer")
     .add_edge("writer", "formatter")
     .add_edge("formatter", END)
     .compile())

 host = AzureAIResponsesAgentHost(
     graph=graph,
     stream_mode="messages",)

 host.run()
```

Full example: [workflows/stage3_foundry_hosted_as_agent.py]

Next steps

Register:
https://aka.ms/AgentsOnFoundry/series

Watch past recordings:
aka.ms/foundryhosted/resources

Join office hours after each session in Discord:
aka.ms/pythonai/oh

Apr 27: **Microsoft Agent Framework**

Apr 29: **LangChain and LangGraph**

Apr 30: **Quality and safety evals**